EN3160 - Assignment 02: Fitting and Alignment

A. N. Wedamestrige | Index: 230687P
Department of Electronic and Telecommunication Engineering, University of Moratuwa
GitHub: https://github.com/AkhilaNisal/EN3160_Assignment_2

Q1. Detecting Circles Using LoG and Scale-Space Extrema

Relevant code

```
sigmas = 2 * 1.15**np.arange(24)
stack = np.array([-s**2 * cv.Laplacian(
    cv.GaussianBlur(f, (0,0), s), cv.CV_32F, ksize=3)
    for s in sigmas])
peaks = (stack == maximum_filter(stack, size=(3,5,5))) & (stack > 0.15)
peaks[0] = peaks[-1] = False
# Each circle radius is sqrt(2) * sigma of its scale-space maximum.
```



Figure 1: Detected sunflower circles (insert the final notebook output).

Results. The notebook reports 174 LoG candidates, 43 appearance-validated candidates and 39 final circles. The tested scale range was $\sigma = 2.00$–49.78; the first and last scale layers were excluded when finding extrema. The five largest reported circles are:

x (px)	y (px)	Radius (px)	σ
282	340	26.47	18.72
0	279	23.02	16.27
177	333	20.01	14.15
65	284	17.40	12.31
107	259	17.40	12.31

Interpretation. The larger detections generally occur near the foreground, while smaller and overlapping flower heads are harder to separate. Some detections extend beyond the image border (e.g., $x = 0$). The detected radius describes the characteristic image blob rather than the exact petal boundary. The saved notebook additionally applies color/appearance filtering after the required LoG extrema stage.

Q2. RANSAC Line and Circle Fitting

Relevant code

```
# RANSAC line: hypothesis from two randomly chosen points
p1, p2 = X[np.random.choice(len(X), 2, replace=False)]
dx, dy = p2-p1
norm = np.hypot(dx, dy)
a, b = dy/norm, -dx/norm
d = a*p1[0] + b*p1[1]
line_inliers = np.abs(a*X[:,0] + b*X[:,1] - d) < 1.5

# Refine line consensus by total least squares
U = X_ - X_.mean(axis=0)
_, _, Vt = np.linalg.svd(U.T @ U)
a, b = Vt[-1]

# Refine RANSAC circle using its radial residuals
result = minimize(objective, circle_model)
best_model_circle = result.x
```

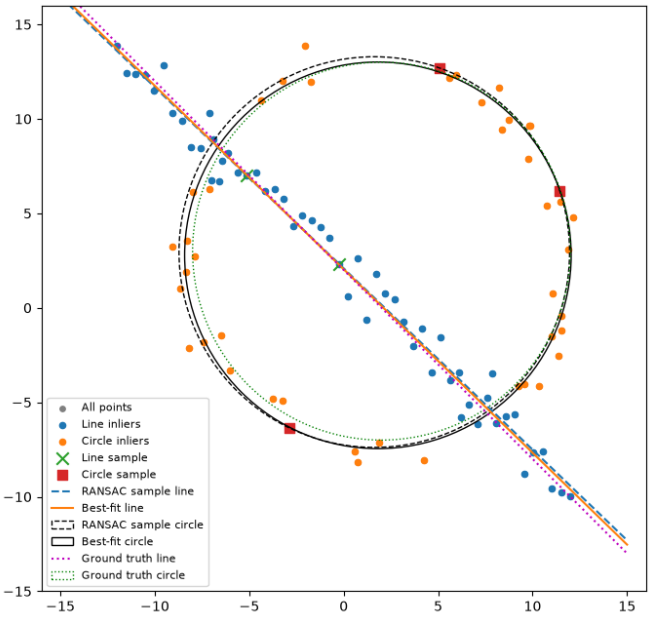


Figure 2: Line/circle samples, inliers, RANSAC fits and ground-truth models.

Results. Using 1000 RANSAC trials, the final line was approximately $0.6968x + 0.7172y = 1.4633$ with 55 line inliers. The circle was estimated as $(x_c, y_c, r) = (1.810, 2.778, 10.231)$, close to the ground truth $(2, 3, 10)$. The circle threshold was 1.2 pixels and the line threshold was 1.5 pixels.

Interpretation (a-d). The normalized line coefficients satisfy $a^2 + b^2 = 1$. More than 50 points were classified as line inliers because some circle points also lie close to the fitted line. The circle remains close to the generated model after removing these points. Fitting the circle first can work, but mixed line/circle samples and points near their intersections may reduce consensus reliability.

Q3. Four-Point Homography and Superimposition

Relevant code

```
H = cv.getPerspectiveTransform(src, dst)
warped = cv.warpPerspective(source, H, (width, height))
mask = cv.warpPerspective(np.ones((h,w),np.uint8)*255,
                          H, (width,height))
blended = cv.addWeighted(target, 0.1, warped, 0.9, 0)
result = target.copy()
result[mask > 0] = blended[mask > 0]
```



Figure 3: Image pair 1: poster superimposed on a billboard.



Figure 4: Image pair 2: NASA graphic superimposed on another billboard.

Interpretation. The two outputs show the source designs following the perspective of their selected billboard surfaces. The warped mask limits blending to the chosen planar region, preserving the surrounding scene. Small visual mismatches can arise from imperfect corner selection, lighting differences and the constant blending weight.

Q4. SIFT-Based PCB Alignment and Difference Images

Relevant code

```
kp1, des1 = sift.detectAndCompute(gray_ref, None)
kp2, des2 = sift.detectAndCompute(gray_test, None)
good = [m for m,n in bf.knnMatch(des2,des1,k=2)
        if m.distance < 0.75*n.distance]
src = np.float32([kp2[m.queryIdx].pt for m in good]).reshape(-1,1,2)
dst = np.float32([kp1[m.trainIdx].pt for m in good]).reshape(-1,1,2)
H, inliers = cv.findHomography(src, dst, cv.RANSAC, 5.0)
aligned = cv.warpPerspective(test, H, (w,h))
diff = cv.absdiff(template, aligned)
```

Results. The reference PCB was `AECC1.jpg`.

Test image	Good matches	RANSAC inliers
AAAA1.jpg	51	18
ADAD1.jpg	369	194
AECC2.jpg	4322	3847
AECC3.jpg	5629	5006
AECC4.jpg	3830	3402



Figure 5: Difference images for the five test PCBs.

Interpretation (a–e). AECC2–4 have many consistent matches, supporting reliable planar alignment. The 18 inliers for AAAA1 suggest its alignment should be interpreted cautiously. Residual differences around the USB port, capacitors and reset button are expected because these elevated parts show parallax that a single planar homography cannot correct. Reflections, shadows and exposure changes add differences; black warp borders in the notebook also create artificial differences.